

Multi-Agent Document & PPT Generation Chatbot

An enterprise-grade **multi-agent AI chatbot** for document analysis, enterprise RAG, real-time web research, editable DOCX/PPTX generation, conversational editing, document conversion, validation, citations, traceability, and version management.

Gemini • FastAPI • Streamlit • Pinecone • FastEmbed • Tavily • Tesseract OCR • python-docx • python-pptx

Overview

The **Multi-Agent Document & PPT Generation Chatbot** is an enterprise-grade Proof of Concept designed to understand natural-language user requests, analyze uploaded documents and presentation templates, perform real-time web research, retrieve information from enterprise knowledge sources, and generate professional editable documents and presentations.

The system follows a specialized **multi-agent architecture** where different agents are responsible for orchestration, document analysis, PPT analysis, web research, enterprise RAG, document generation, PPT generation, validation, and conversational artifact editing.

Users can upload existing **PDF, DOCX, PPT/PPTX, PNG, JPG, and JPEG** files and interact with the system through natural-language requests.

The generated artifacts can subsequently be edited through conversational commands while maintaining their existing structure and context.

Assignment Objective

The project was developed to satisfy the requirements of the **Multi-Agent AI Chatbot for Document & PPT Generation** assignment.

The system demonstrates:

-  Multi-agent orchestration
 -  Document analysis
 -  PPT template analysis
 -  Real-time web research
 -  Enterprise Retrieval-Augmented Generation (RAG)
 -  Pinecone vector database integration
 -  OCR processing for image/scanned content
 -  Editable DOCX generation
 -  Editable PPTX generation
 -  Template style and structure preservation
 -  Conversational artifact editing
 -  DOCX ↔ PPTX conversion
 -  Source citations and traceability
 -  Artifact version management
 -  Generated artifact validation
 -  Modular backend APIs
-

☆☆ Key Features

🔧 1. Multi-Agent Architecture

The application uses specialized agents instead of implementing the entire workflow inside a single LLM call.

- 🧑 Supervisor / Orchestrator Agent
- 📄 Document Analyzer Agent
- 📊 PPT Analyzer Agent
- 🌐 Web Research Agent
- 🧠 RAG Agent
- 📝 Document Generation Agent
- 📊 PPT Generation Agent
- ✅ Validation Agent
- 💬 Conversational Editing Agent

The Supervisor determines which agents are required for each user request and coordinates the execution pipeline.

📄 2. Document Analysis

The system can analyze uploaded documents and extract useful structural and content information including:

- Document type
- Tone
- Heading hierarchy
- Fonts
- Primary font
- Paragraph information
- Tables
- Section titles
- Title information
- Content structure

Supported document inputs include:

- PDF
 - DOCX
 - Scanned documents
 - Images
-

📊 3. PPT Template Analysis

Existing PowerPoint templates can be uploaded and analyzed to understand presentation-level formatting and design characteristics.

The system considers:

- Slide dimensions
- Visual style
- Primary fonts
- Layout information
- Slide structure
- Content patterns

- Presentation formatting

Generated presentations use the analyzed template information to maintain the overall visual style and structure.

🌐 4. Real-Time Web Research

The Web Research Agent uses **Tavily** to perform real-time web research when the user's request requires current or recent information.

Example request:

```
Research the latest developments in Generative AI and
create a concise report with sources.
```

The research workflow is:

```
User Query
  ↓
Supervisor
  ↓
Web Research Agent
  ↓
Tavily Search
  ↓
Relevant Web Sources
  ↓
Research Findings
  ↓
Content Planning / Generation
```

Web research results are incorporated into generated content and associated with source citations.

🏢 5. Enterprise RAG

The application implements Retrieval-Augmented Generation for retrieving relevant information from uploaded enterprise documents.

The RAG pipeline is:

```
Uploaded Document
  ↓
Text Extraction
  ↓
Chunking
  ↓
FastEmbed Embeddings
  ↓
Pinecone Vector Database
  ↓
Semantic Similarity Search
  ↓
Relevant Chunks
  ↓
LLM / Content Generation
```

FastEmbed is used for local embedding generation and **Pinecone** is used as the enterprise vector database.

The system also supports a local FAISS vector-store fallback for development and environments where Pinecone is unavailable.

🔗 6. Source Citations & Traceability

The application maintains source information throughout the pipeline. Retrieved content can be associated with citation identifiers such as:

RAG-001
RAG-002
WEB-001
WEB-002

Each orchestration request generates a trace identifier that allows the execution flow to be inspected.

Example:

Trace ID: trace_d8cd9d501e

The trace can contain information about:

- User request
- Execution plan
- Agents executed
- Document analysis
- PPT analysis
- Web research
- RAG retrieval
- Citations
- Generated artifact IDs
- Validation results
- Warnings

7. OCR / Image Processing

The system supports scanned and image-based content through OCR processing.

Image formats supported include:

- PNG
- JPG
- JPEG

The OCR pipeline uses **Tesseract OCR** through the **pytesseract** Python library.

```
Image / Scanned Document
  ↓
OCR Engine
  ↓
Extracted Text
  ↓
Document Analysis
  ↓
RAG / LLM
  ↓
Response
```

The system also supports OCR fallback for scanned PDFs when selectable text cannot be extracted.

8. Editable DOCX Generation

The Document Generation Agent generates real editable Microsoft Word documents using **python-docx**.

Generated reports can contain:

- Title
- Executive Summary
- Main Sections

- Key Findings
- Important Details
- Conclusion
- Sources / Citations

The generated output is an actual editable **.docx** file rather than a PDF or image representation.

9. Editable PPTX Generation

The PPT Generation Agent creates editable PowerPoint presentations using **python-pptx**.

The content is adapted to presentation format instead of simply copying large document paragraphs onto slides.

Generated presentations can contain:

- Title slide
- Overview
- Main sections
- Key points
- Important details
- Conclusion
- Sources

Dense sections can automatically be divided across multiple slides.

10. Template-Based Generation

The system supports generation using uploaded templates.

For example:

```
Uploaded PDF
+
Uploaded DOCX Template
↓
Document Analysis
↓
Content Planning
↓
Generated Editable DOCX
```

Similarly:

```
Uploaded PDF
+
Uploaded PPTX Template
↓
PPT Analysis
↓
Content Planning
↓
Generated Editable PPTX
```

The system attempts to preserve the template's overall:

- Structure
 - Tone
 - Formatting
 - Visual style
 - Content organization
-

11. Conversational Artifact Editing

Generated artifacts can be modified through natural-language instructions.

Examples:

Add an executive summary.

Make the presentation more concise.

Add a competitive analysis section.

Update the report using the latest web information.

The Conversational Editing Agent loads the existing content plan, applies the requested modification, generates a new version, and preserves the artifact's existing context.

12. Version Management

Generated artifacts maintain version history.

Example:

Document
├ v1
└ v2

Each version maintains associated content-plan information and metadata.

This allows users to track changes made through conversational editing.

13. Document / Presentation Conversion

The application supports conversion between document and presentation formats.

DOCX → PPTX

PPTX → DOCX

The conversion pipeline adapts the content to the target format instead of simply renaming or copying the source file.

14. Artifact Validation

Generated artifacts are validated before being presented as successful outputs.

DOCX Validation

- File can be opened
- Expected structure exists
- Document contains body content
- Required sections can be checked
- Citations are validated

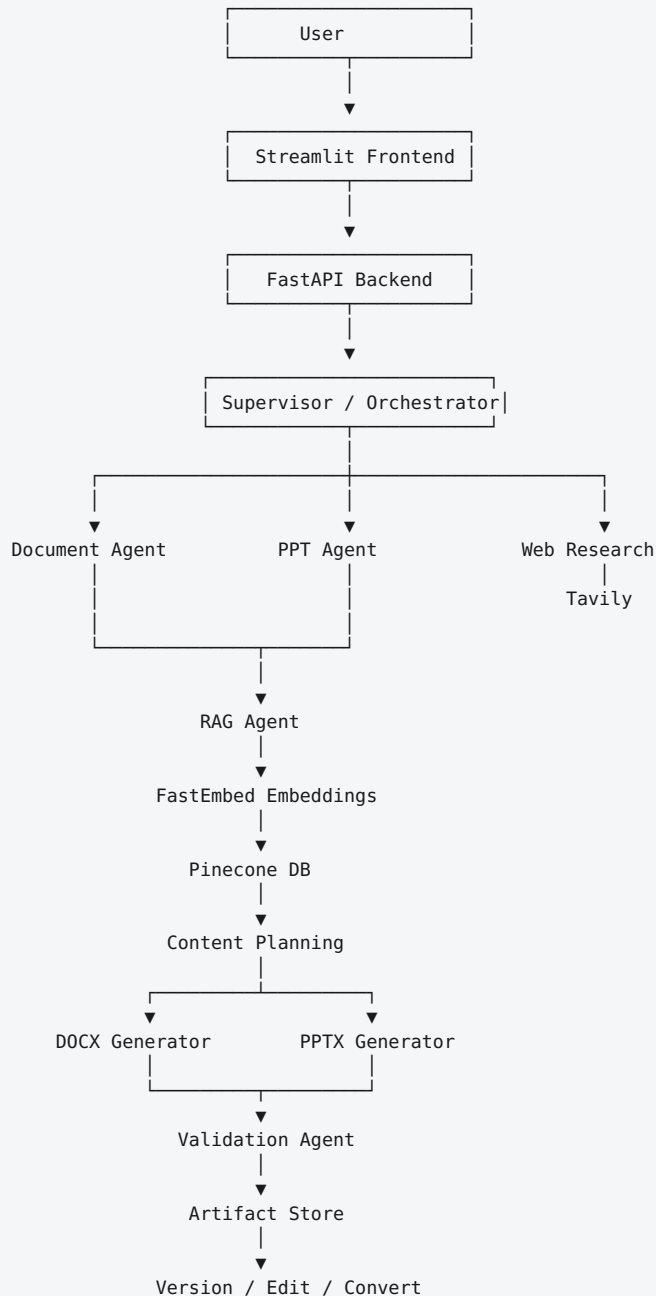
PPTX Validation

- File can be opened
- Slides are structurally valid

- Slides are not empty
- Requested slide count can be validated when applicable
- Citations are validated

Validation status is displayed in the Streamlit artifact panel.

System Architecture



Multi-Agent Workflow

1. User uploads documents/templates and enters a natural-language request.
2. The Supervisor Agent analyzes the request and creates an execution plan.
3. The Document Analyzer processes PDF/DOCX files when required.

4. The PPT Analyzer processes uploaded PPT/PPTX templates when required.
5. The Web Research Agent performs real-time research through Tavily when current information is requested.
6. The RAG Agent retrieves relevant enterprise knowledge from Pinecone.
7. The Content Planner combines the request, research findings, retrieved knowledge, and template information.
8. The Document Generator creates an editable DOCX when requested.
9. The PPT Generator creates an editable PPTX when requested.
10. The Validation Agent verifies the generated artifact.
11. The Artifact Store saves the generated artifact and its version metadata.
12. The user can subsequently modify the artifact using natural-language instructions.

Technology Stack

Category	Technology
Programming Language	Python
Frontend	Streamlit
Backend	FastAPI
ASGI Server	Uvicorn
LLM	Google Gemini
Web Research	Tavily
Embeddings	FastEmbed
Vector Database	Pinecone
Local Vector Store / Fallback	FAISS
PDF Processing	PyMuPDF
DOCX Processing	python-docx
PPTX Processing	python-pptx
OCR	Tesseract + pytesseract
Image Processing	Pillow
Configuration	python-dotenv / pydantic-settings
Testing	pytest

Project Structure

```
docgen-multiagent_Phase16/
├── backend/
│   ├── agents/
│   │   ├── supervisor.py
│   │   ├── document_analyzer.py
│   │   ├── ppt_analyzer.py
│   │   ├── web_research.py
│   │   ├── rag_agent.py
│   │   ├── document_generator.py
│   │   ├── ppt_generator.py
│   │   ├── conversational_editor.py
│   │   └── converter.py
│   ├── api/
│   │   ├── chat.py
│   │   ├── ingest.py
│   │   ├── generate.py
│   │   ├── convert.py
│   │   └── trace.py
│   ├── services/
│   │   ├── llm_service.py
│   │   ├── extraction_service.py
│   │   ├── embedding_service.py
│   │   ├── vector_store.py
│   │   ├── ocr_service.py
│   │   ├── artifact_store.py
│   │   └── citation_service.py
│   ├── schemas/
│   │   └── main.py
│   └── frontend/
│       └── app.py
├── data/
│   ├── samples/
│   │   ├── templates/
│   │   │   └── report_template.docx
│   │   └── outputs/
│   │       ├── sample_report.docx
│   │       └── sample_presentation.pptx
│   ├── generated/
│   ├── uploads/
│   ├── templates/
│   ├── versions/
│   └── knowledge_base/
├── tests/
├── .env.example
├── .gitignore
├── requirements.txt
└── README.md
```

⚙️ Installation

1. Clone the Repository

```
git clone <YOUR_GITHUB_REPOSITORY_URL>
cd docgen-multiagent_Phase16
```

2. Create Virtual Environment

Windows

```
python -m venv venv
venv\Scripts\activate
```

Linux / macOS

```
python3 -m venv venv
source venv/bin/activate
```

3. Install Python Dependencies

```
pip install -r requirements.txt
```

4. Configure Environment Variables

Create a `.env` file from `.env.example`.

```
copy .env.example .env
```

Configure the required API keys and providers inside `.env`.

🔑 Environment Configuration

Typical configuration:

```
LLM_PROVIDER=gemini
GEMINI_API_KEY=YOUR_GEMINI_API_KEY

VECTOR_STORE=pinecone
PINECONE_API_KEY=YOUR_PINECONE_API_KEY
PINECONE_INDEX_NAME=docgen-multiagent

WEB_SEARCH_PROVIDER=tavily
TAVILY_API_KEY=YOUR_TAVILY_API_KEY
```

Never commit the `.env` file or API keys to GitHub.

📄 Tesseract OCR Setup

The Python package `pytesseract` requires the Tesseract OCR engine to be installed separately.

Windows

1. Install Tesseract OCR.
2. Add the Tesseract installation directory to PATH.
3. Restart the terminal.

Verify installation:

```
tesseract --version
```

If required, configure the executable path in the OCR service.

▶ Running the Application

Start the FastAPI Backend

Open Terminal 1:

```
venv\Scripts\activate
uvicorn backend.main:app --reload --port 8000
```

The backend will be available at:

```
http://127.0.0.1:8000
```

Start the Streamlit Frontend

Open Terminal 2:

```
venv\Scripts\activate
streamlit run frontend/app.py
```

The Streamlit interface will normally be available at:

```
http://localhost:8501
```

How to Use

Step 1 — Upload Files

Upload one or more supported files through the Streamlit sidebar.

- PDF
- DOCX
- PPT
- PPTX
- PNG
- JPG
- JPEG

Step 2 — Enter a Request

Example:

```
Research the latest Generative AI trends and create a
proposal and presentation using the uploaded templates.
```

Step 3 — Agent Execution

The application automatically determines which agents are needed.

Step 4 — Review Results

The UI displays agent activity, citations, trace information, generated artifacts, and validation status.

Step 5 — Edit the Artifact

Select the generated artifact and enter a natural-language modification.

Example:

```
Add a short Key Takeaways section at the end of the document.
```

A new artifact version is generated.

Example Assignment Workflow

The assignment provides the following type of workflow:

User uploads:

Company_Proposal.docx
Company_Template.pptx

User asks:

"Research the latest Generative AI trends and create a proposal and 12-slide presentation using the same tone and style as the uploaded files."

The system performs:

1. Analyze uploaded templates
2. Perform real-time web research
3. Retrieve enterprise knowledge using RAG
4. Generate document and presentation
5. Validate generated content
6. Provide citations and sources
7. Return editable DOCX and PPTX files

The user can then continue with requests such as:

- Add an executive summary.
- Make the presentation more concise.
- Add a competitive analysis section.
- Update the report using the latest web information.

Artifact Editing Workflow

```
Generated Artifact
  ↓
User Edit Request
  ↓
Conversational Editing Agent
  ↓
Load Existing Content Plan
  ↓
Apply Requested Changes
  ↓
Generate New Version
  ↓
Validate
  ↓
Save Version
```

Example:

```
v1
  ↓
"Add Key Takeaways"
  ↓
v2
```

DOCX ↔ PPTX Conversion

The application provides document/presentation conversion capabilities.

```
DOCX → PPTX
```

```
PPTX → DOCX
```

The conversion process adapts content to the target format and stores the result as a separate artifact.

Backend API

Endpoint	Purpose
GET /health	Check backend health and configured providers
POST /upload	Upload supported files
POST /ingest/{file_id}	Extract and normalize uploaded file content
POST /chat	Run the multi-agent orchestration pipeline
POST /generate/document	Generate an editable DOCX
POST /generate/presentation	Generate an editable PPTX
GET /trace/{trace_id}	Retrieve orchestration trace information
POST /validate/{artifact_id}	Validate a generated artifact

Testing

The project includes automated tests using **pytest**.

```
pytest -v
```

Individual test modules can also be executed independently.

```
pytest tests/test_rag.py -v
```

```
pytest tests/test_web_research.py -v
```

```
pytest tests/test_document_generation.py -v
```

```
pytest tests/test_ppt_generation.py -v
```

```
pytest tests/test_validation.py -v
```

Sample Templates & Generated Outputs

Sample assignment artifacts are included in:

```

data/
├── samples/
│   ├── templates/
│   │   └── report_template.docx
│   └── outputs/
│       ├── sample_report.docx
│       └── sample_presentation.pptx

```

These files demonstrate template-based generation and editable output generation.

📁 Assignment Requirement Coverage

Assignment Requirement	Status
Multi-agent chatbot with Supervisor/Orchestrator	✔ Implemented
Document and PPT template analysis	✔ Implemented
Real-time web search and research	✔ Implemented
Enterprise RAG with vector database	✔ Pinecone
PDF support	✔ Implemented
DOCX support	✔ Implemented
PPT/PPTX support	✔ Implemented
Image/scanned document support	✔ Implemented
OCR/vision processing	✔ Tesseract OCR
Editable DOCX generation	✔ Implemented
Editable PPTX generation	✔ Implemented
Template tone and structure preservation	✔ Implemented
Template formatting / visual style	✔ Implemented
Conversational editing	✔ Implemented
DOCX ↔ PPTX conversion	✔ Implemented
Source citations and traceability	✔ Implemented
Version management	✔ Implemented
Secure/modular backend APIs	✔ Implemented
Sample templates and outputs	✔ Included
Complete documentation	📖 README / Documentation
requirements.txt	✔ Included

⚠ Configuration & Limitations

- API keys for Gemini, Pinecone, and Tavily must be configured in `.env`.
- Tesseract OCR requires the system-level Tesseract executable in addition to the Python `pytesseract` package.

- The first FastEmbed model initialization may require downloading the embedding model.
- Real-time web research requires a valid Tavily API key.
- Pinecone functionality requires a valid Pinecone API key and configured index.

Security

- API keys are loaded through environment variables.
- `.env` is excluded from Git.
- Uploaded/generated runtime data is excluded from Git.
- Backend functionality is separated into modular APIs and services.
- Generated artifacts are validated before being reported as successful.

Deployment

The project is designed to be deployed through GitHub for demonstration and evaluation purposes.

The repository should contain:

- Complete source code
- requirements.txt
- .env.example
- README.md
- Sample templates
- Sample generated outputs
- Automated tests

Sensitive credentials must never be committed to the repository.

Demonstration Workflow

A recommended demonstration sequence is:

1. Start FastAPI backend.
2. Start Streamlit frontend.
3. Upload a PDF/document.
4. Upload a DOCX template.
5. Upload a PPTX template.
6. Ask for research + proposal + presentation.
7. Show document analysis.
8. Show PPT template analysis.
9. Show Tavily research.
10. Show Pinecone RAG retrieval.
11. Show generated DOCX.
12. Show generated PPTX.
13. Show validation status.
14. Edit the generated document.
15. Show v1 → v2 version history.
16. Convert DOCX ↔ PPTX.
17. Show traceability information.

Future Improvements

- Production-grade authentication and authorization
 - Persistent conversation management
 - Advanced template style cloning
 - More sophisticated layout intelligence
 - Distributed task execution
 - Cloud deployment and monitoring
 - Advanced observability dashboards
-

Author

Vedant Gaikwad

Electronics & Telecommunication Engineering

AI / Machine Learning / Generative AI

Assignment Deliverables

The project is organized to provide the deliverables requested in the assignment:

-  Fully functional multi-agent chatbot POC
-  Complete source code
-  Document/PPT template analysis
-  Web research
-  Enterprise RAG
-  Editable DOCX generation
-  Editable PPTX generation
-  Conversational artifact editing
-  Source/citation traceability
-  Sample templates
-  Sample generated outputs
-  Project documentation
-  README setup instructions
-  Usage guidelines
-  requirements.txt

The assignment also recommends GitHub deployment and a working project demonstration video.

Built with Python • FastAPI • Streamlit • Gemini • Pinecone • FastEmbed • Tavily • Tesseract OCR

 Multi-Agent AI •  Document Intelligence •  Presentation Generation •  Enterprise RAG •  Web Research

Made with ♥ for the Multi-Agent AI Document & PPT Generation Assignment